



## Assignment 1

Aditya Fadhillah (hig708)

TA: Therese Lyngby

Program Analysis and Transformation

Department of Computer Science, May 10, 2025

## Program Inversion and Inverse Interpretation

### 1. Define a trivial program inverter given an inverse interpreter

Given an inverse interpreter `invint`, the program inverter `inverter` is defined as:

$$\text{inverter} = \lambda p. \lambda y. \text{invint}(p, y)$$

Meaning that, if  $p(x) = y$ , then  $\text{inverter}(p)(y) = x$ . The inverter recovers the original input by applying the inverse interpreter to the program and its output.

### 2. Define a trivial inverse interpreter given a program inverter

Given a program inverter `inverter`, the inverse interpreter `invint` is defined as:

$$\text{invint} = \lambda(p, y). \text{inverter}(p)(y)$$

This defines the inverse interpreter as a function of a program  $p$  and an output  $y$ , returning the corresponding input by applying the inverter.

## Reversible Programming

### 3. Run the Janus-program fib forward by hand

Program:

```
x1 += 1
x2 += 1
from x1 = x2 loop
  n -= 1
  x1 += x2
  x1 <=> x2
until n = 0
```

-----

Initial store:

`n = 4, x1 = 0, x2 = 0`

| Statement           | n | x1 | x2 |
|---------------------|---|----|----|
| x1 += 1             | 4 | 1  | 0  |
| x2 += 1             | 4 | 1  | 1  |
| from x1 = x2 (true) | 4 | 1  | 1  |
| n -= 1              | 3 | 1  | 1  |
| x1 += x2            | 3 | 2  | 1  |
| x1 <=> x2           | 3 | 1  | 2  |
| until n = 0 (false) | 3 | 1  | 2  |
| n -= 1              | 2 | 1  | 2  |
| x1 += x2            | 2 | 3  | 2  |
| x1 <=> x2           | 2 | 2  | 3  |
| until n = 0 (false) | 2 | 2  | 3  |
| n -= 1              | 1 | 2  | 3  |
| x1 += x2            | 1 | 5  | 3  |
| x1 <=> x2           | 1 | 3  | 5  |
| until n = 0 (false) | 1 | 3  | 5  |
| n -= 1              | 0 | 3  | 5  |
| x1 += x2            | 0 | 8  | 5  |
| x1 <=> x2           | 0 | 5  | 8  |
| until n = 0 (true)  | 0 | 5  | 8  |

Final store:

`n = 0, x1 = 5, x2 = 8`

#### 4. Run the Janus-program fib backward by hand

Reversed program:

```

from n = 0 loop
  x1 <=> x2
  x1 -= x2
  n += 1
until x1 = x2
x2 -= 1
x1 -= 1

```

Initial store:

n = 0, x1 = 5, x2 = 8

| Statement             | n | x1 | x2 |
|-----------------------|---|----|----|
| from n = 0 (true)     | 0 | 5  | 8  |
| x1 <=> x2             | 0 | 8  | 5  |
| x1 -= x2              | 0 | 3  | 5  |
| n += 1                | 1 | 3  | 5  |
| until x1 = x2 (false) | 1 | 3  | 5  |
| x1 <=> x2             | 1 | 5  | 3  |
| x1 -= x2              | 1 | 2  | 3  |
| n += 1                | 2 | 2  | 3  |
| until x1 = x2 (false) | 2 | 2  | 3  |
| x1 <=> x2             | 2 | 3  | 2  |
| x1 -= x2              | 2 | 1  | 2  |
| n += 1                | 3 | 1  | 2  |
| until x1 = x2 (false) | 3 | 1  | 2  |
| x1 <=> x2             | 3 | 2  | 1  |
| x1 -= x2              | 3 | 1  | 1  |
| n += 1                | 4 | 1  | 1  |
| until x1 = x2 (true)  | 4 | 1  | 1  |
| x2 -= 1               | 4 | 1  | 0  |
| x1 -= 1               | 4 | 0  | 0  |

Final store:

n = 4, x1 = 0, x2 = 0

#### 5. Describe informally each statement

The Janus program for iterative Fibonacci uses several constructs of reversible programming. Below is a simple explanation of each construct and how it behaves in forward and backward execution:

- **Reversible assignments (+=) (-=):** These update a variable by adding or subtracting a value. For example, `x1 += 1` increases `x1` by 1 in forward execution. In reverse, this becomes `x1 -= 1`. These operations are reversible because no information is lost.
- **Swap (`x1 <=> x2`):** This swaps the values of two variables. It is its own inverse, meaning the same statement undoes itself when run in reverse. Statements like `x1 <=> x2`, `+=`, and `-=` are atomic statements in Janus.
- **Reversible loop (from-loop-until):** This is Janus's reversible version of a while-loop. The **from** condition is checked once when entering the loop in forward execution and acts as an entry assertion; if false, the program aborts. In the Fibonacci program, **from** `x1 = x2` holds before the first iteration. After each execution of the loop body, the program must ensure (semantically) that `x1 = x2` becomes false. This is not actively checked in forward execution, but is required to guarantee reversibility. The **until** condition acts as the loop's exit condition in forward execution. If any assertion fails when expected, the program aborts.
- **Sequence of statements:** Inside the loop, the statements are executed in order. When reversing, Janus automatically runs them in the opposite order, reversing each one.

These constructs let Janus run programs both forward and backward. In the Fibonacci example, this means that it is possible to compute values step by step and also reverse those steps to recover the input.

## 6. Describe the semantics (natural semantics)

The following describes the semantics of the Janus constructs used in the `fib` program.

**Reversible assignment:** `x += e`

$$\begin{aligned}\text{Eval}(x + = e, \sigma) &= \sigma[x \mapsto \sigma(x) + \text{Eval}(e, \sigma)] \\ \text{Eval}^{-1}(x + = e, \sigma) &= \text{Eval}(x - = e, \sigma) = \sigma[x \mapsto \sigma(x) - \text{Eval}(e, \sigma)]\end{aligned}$$

**Swap statement:** `x <=> y`

$$\text{Eval}(x <=> y, \sigma) = \sigma[x \mapsto \sigma(y), y \mapsto \sigma(x)]$$

This operation is its own inverse.

**Reversible loop:** `from e1 loop s until e2`

$$\text{Eval}(\text{from } e_1 \text{ loop } s \text{ until } e_2, \sigma) = \begin{cases} \text{abort} & \text{if } \text{Eval}(e_1, \sigma) \neq \text{true} \\ \text{repeat: } \begin{array}{l} \sigma := \text{Eval}(s, \sigma) \\ \text{assert } \text{Eval}(e_1, \sigma) = \text{false} \end{array} & \text{while } \text{Eval}(e_2, \sigma) \neq \text{true} \end{cases}$$

$$\text{Eval}^{-1}(\text{from } e_1 \text{ loop } s \text{ until } e_2, \sigma) = \begin{cases} \text{abort} & \text{if } \text{Eval}(e_2, \sigma) \neq \text{true} \\ \text{repeat: } \begin{array}{l} \sigma := \text{Eval}^{-1}(s, \sigma) \\ \text{assert } \text{Eval}^{-1}(e_2, \sigma) = \text{false} \end{array} & \text{while } \text{Eval}(e_1, \sigma) \neq \text{true} \end{cases}$$

The `from` condition acts as an entry assertion (checked once before the first iteration), and the `until` condition is the loop's exit condition. To guarantee reversibility, the program must ensure that after every execution of the loop body `s`, `Eval(e1, σ) = false` holds.

**Sequence of statements:** `s1; s2`

$$\text{Eval}(s_1; s_2, \sigma) = \text{let } \sigma' = \text{Eval}(s_1, \sigma) \text{ in } \text{Eval}(s_2, \sigma')$$

$$\text{Eval}^{-1}(s_1; s_2, \sigma) = \text{let } \sigma' = \text{Eval}^{-1}(s_2, \sigma) \text{ in } \text{Eval}^{-1}(s_1, \sigma')$$